

Compilation course

[Lexical and syntax analysis](#) / Lab 2 - Tiger Calculator

In this lab we will study and improve a Lexer and Parser for the Tiger language. We will also write an AST Iterator that evaluates simple integer Tiger expressions.

We will start from a compiler skeleton built by your teachers.

Import the example code

Retrieve the lab code and commit it to your git with the following commands in the subdirectory `lab2` at the root of your git repository.

Execute the following commands at the top of your repository:

```
$ mkdir lab2
$ cd lab2
$ wget -qO- www.siffle.org/lectures/compil/lab2/dragon-tiger.tar.gz | tar zxv
$ git add dragon-tiger
$ git commit -m "Import dragon-tiger for lab2" dragon-tiger
```

Look around

The sources in `dragon-tiger` are organized this way:

- * `src/ast` contains the definition of the tree. In `nodes.hh` you can see all the nodes declaration. You should not alter this file.
- * `src/parser` contains the lexer (`tiger_lexer.ll`) for use with the `lex/flex` lexer generator, and the parser (`tiger_parser.yy`) for the `yacc/bison` parser generator.
- * `src/driver` contains the main program in `driver.cc`.
- * `src/utlis` contains some utility classes to deal with errors.

If needed, `flex` documentation is [located here](#) and `bison` documentation can be [found here](#), but the existing code should be self-explanatory and reading it should be enough to implement the requested features.

The AST nodes

The AST contains nodes derived from the `Node` class. There are two kind of nodes, which derive directly from this top-level `Node` class: `Expr` represents any form of Tiger expressions, while `Decl` represents declarations.

The `nodes.hh` also contains structures that help use the [visitor pattern](#). You should read and understand it, as the last item of this lab will need to use it.

Building the project

From within the `dragon-tiger` repository, run:

```
$ ./configure
$ make
```

If everything worked as expected, you should be able to run the compiler driver `dtiger` as follows,

```
$ src/driver/dtiger --help
Options:
  -h [ --help ]           describe arguments
  --dump-ast              dump the parsed AST
  --trace-parser          enable parser traces
  --trace-lexer           enable lexer traces
  -v [ --verbose ]        be verbose
  --input-file arg        input Tiger file

$ echo "a * b * c * d" > test.tig
$ src/driver/dtiger --dump-ast test.tig
(a*(b*(c*d)))
```

You can use `-` as a file name to read the Tiger code from the standard input without using an intermediate file:

```
$ echo "a * b" | src/driver/dtiger --dump-ast -
```

```
(a*b)
```

As you can see a simple lexer, parser, and AST dumper (printer) are already implemented.

In the following lab you will add new features to `dtiger`, ensure that you test thoroughly and commit each feature. Follow precisely the instructions, this lab is graded and machine corrected!

Remember: you should never have to modify `ast/nodes.hh` during this lab.

Supporting integers

Your first task is implementing support for integer literals.

```
$ echo "1*2" | src/driver/dtiger --dump-ast -
1.1: invalid character
```

As you can see in the above example, currently, the lexer and parser do not recognize integers. To debug the parser and lexer you can activate trace modes:

```
$ echo "1*2" | src/driver/dtiger --dump-ast --trace-lexer -
--(end of buffer or a NUL)
--accepting rule at line 122 ("1")
1.1: invalid character
```

(line numbers may vary)

Can you understand what is happening? Look for the string "invalid character" in the lexer file `src/parser/tiger_lexer.ll`. It looks like someone forgot to add a rule to recognize integers.

► First add an `INT` token in the parser. Look at `src/parser/tiger_parser.yy` after the comment starting with "Define tokens". The type of the token should be `int`, its name `INT`, and its comment `integer`.

► Add support for recognizing integers in the lexer. You should look at how this is implemented for `ID` tokens. First, add a rule for a regular expression matching integers. No need to worry about the minus sign; it is already handled in the parser. In the `flex` action for integers, use the function `strtol` to convert the matched text (`yytext`) to an integer and emit an `INT` token. You should ensure that the parsed integer is in the range of legal Tiger integers (the constant `TIGER_INT_MAX` has been defined for you), and signal a correctly placed error using `utils::error` if not.

► Ensure that you reject numbers with leading 0 as the Tiger language forbids them (but you must accept 0 of course).

Now the lexer should recognize integers,

```
$ echo "1*2" | src/driver/dtiger --trace-lexer --trace-parser --dump-ast -
Starting parse
Entering state 0
Reading a token: --(end of buffer or a NUL)
--accepting rule at line 87 ("1")
Next token is token "int" (1.1: )
1.1: syntax error, unexpected int
```

but the parser still complains because integers are not valid expressions.

► Add support for integer expressions in the parser. You should add a new rule that matches an `INT` token and returns an AST Node of type `IntegerLiteral`. For this, you need to perform three steps: add an `intExpr` rule doing the matching and returning a new `IntegerLiteral` node, declare the type of `intExpr` as being `Expr *` (see how this is done for `stringExpr` for example), and add a case to the `expr` rule to make it accept `intExpr` everywhere an `expr` is acceptable (as it does for `stringExpr` for example).

When you finish this section, the Tiger program `1*2` should be properly parsed!

A note on symbols

Throughout the lexer and parser files, you may have noticed that strings are represented using the `Symbol` type. This type, which is defined in `src/utills/symbols.hh`, allows to detect when similar objects are used (here we use it for strings and identifiers) and return a shared reference for similar objects.

For example, if the `foobar_identifier_123` identifier is used 10,000 times in your Tiger source code, the string will be present only once in memory and the AST will contain 10,000 identical references to it instead of pointing onto 10,000 different copies.

Adding support for more binary operators and adding precedence rules

Before diving in the subject of precedence rules, let's have a look at how we handle the *unary minus* operation. We could add a node `UnaryMinus` in our AST, or

we could take advantage that $-x$ holds the same value as $0-x$, and we already know how to represent a subtraction in our AST using a `BinaryOperator` node. Hence this is what we are choosing here: -4 will produce an AST of $(0-4)$. Hopefully, this subtraction will be optimized in a further step by the compiler and will be performed at compile time rather than at runtime every time the program is run.

Let us now worry about operator precedence. Currently the following expression $1*2+3$ is parsed as $(1*(2+3))$. This is clearly incorrect, because the $*$ is more binding than the $+$.

If you look at the file `src/parser/bison-report.txt` you will find many shift/reduce and reduce/reduce conflicts. Such as,

```
State 25

20 negExpr: "-" expr .
21 opExpr: expr . "+" expr
22         | expr . "-" expr
23         | expr . "*" expr
24         | expr . "/" expr
25         | expr . "=" expr
26         | expr . "<" expr
27         | expr . "<" expr
28         | expr . ">" expr
29         | expr . "<=" expr
30         | expr . ">=" expr
31         | expr . "&" expr

"+" shift, and go to state 32
"-" shift, and go to state 33

[-]

"+" [reduce using rule 20 (negExpr)]
"-" [reduce using rule 20 (negExpr)]

[-]
```

Here the parser is parsing the following kind of construct $-4+5$ and got confused. Should it be parsed as $-(4+5)$ (incorrect) or $(-4)+5$ (correct)? The parser does not contain the necessary information yet.

How to read this report? The reading cursor, depicted by a dot ($.$) is currently just before the $+$ operator: it parsed a minus sign $-$, an expression 4 , and does not know what to do when it encounters a $+$ sign. It hesitates between:

- **reducing** (-4) right now which will ultimately produce the following (correct) interpretation $(-4) + 5$ and the AST $((0-4)+5)$
- **shifting** which will ultimately produce the following (incorrect) interpretation $-(4+5)$ and the AST $(0-(4+5))$

The default choice in Bison is to shift. Therefore here, the parser will produce the second incorrect AST.

To fix these ambiguities one must declare precedence rules for the operators and some keywords. If you look for "Declare precedence rules" in `tiger_parser.yy` you will find at this stage

```
%nonassoc FUNCTION VAR TYPE DO OF ASSIGN;
%left UMINUS;
```

It means that `FUNCTION`, `VAR`, and so on are not associative. It also means that the unary minus (denoted by `UMINUS`) is associative left (not that it matters for an unary operator), and that it has a higher precedence than `FUNCTION`, `VAR`, and so on, because it is declared after them.

As an example, $+$ should be associative left, and have a precedence lower ("be declared before") than the unary minus, because you want $-4+5$ to be parsed as $(-4)+5$, not as $-(4+5)$.

► Add precedence rules until you fix all the shift/reduce and reduce/reduce conflicts in the parser. Check that you correctly parse arithmetic expressions such as $1+2*3$, $5-2-1$ or $-4+5$.

Adding support for the boolean OR operator

To simplify subsequent phases in the compiler, boolean operators are translated to an `IfThenElse` AST node. Indeed, due to Tiger's lazy evaluation of boolean operators, $a \ \& \ b$ is semantically equivalent to the following code,

```
if (a) then (if (b) then 1 else 0) else 0
```

► Add support for `|` (OR) boolean operator in the parser. Do not produce a `BinaryOperator` node, use an `IfThenElse` node with a construction similar to the one used by the `&` (AND) operator.

► Test that your compiler recognizes the new constructs.

Adding support for `if then else` constructs

► Add support for `if then else` and `if then` constructs to the lexer and parser.

To simplify subsequent phases in the compiler, when you find a naked

```
if condition then
  body
```

block replace it by the following equivalent construct,

```
if condition then
  body
else
  ()
```

The `()` corresponds to a `Sequence` AST node with an empty expression vector.

Be careful to fix any shift/reduce or reduce/reduce conflicts you may introduce. Do not forget to declare the type of the new expression you created and to add it to the list of acceptable expressions.

Implementing an AST evaluator for simple Tiger int expressions

Now that your parser and lexer are complete, we are going to add a new AST iterator that evaluates them to build a simple Tiger calculator.

The AST evaluator should support evaluating the following AST nodes:

- An `IntegerLiteral` evaluates to its integer value.
- A `BinaryOperator` evaluates to the result between the two operands. For boolean operators `1` is true and `0` is false.
- A `Sequence` evaluates all its sub-expressions, and evaluates to the value of its last sub-expression. An empty sequence should raise an error and terminate the evaluation.
- A `IfThenElse` block evaluates to the value of the `then` or `else` branch depending if the condition is true or false. The other branch is not evaluated at all.

All the other nodes should raise an error when evaluated.

The evaluator can be called with the `-e` (and `--eval`) command line option. It should work as follows,

```
$ echo "2*3" | src/driver/dtiger -e -
6
```

Please ensure that your implementation works exactly as above so that this work can be graded automatically.

When using both `-e` and `--dump-ast`, an error should be triggered.

Errors should be raised with the `util/errors.hh` functions and should be fatal.

The evaluator should reside in the `src/ast` directory in the `ast` namespace and extend the class `ConstASTIntVisitor`.

► Implement the Evaluator as described above.